UdaanHer Saathi MVP: Task Book

Version 1.0 — 26 tasks, in order, each assignable to one person with no other context

Team: Aishi + mentor • 9 July 2026

Contents

1. How to use this book

Every task below follows the same anatomy: **metadata** (what it depends on, how long it should take a careful beginner, what kind of person can own it), **context** (enough background that you need no meeting), **what to do** (concrete steps), and a **definition of done** — a checklist another team member can verify without trusting you. "Spec §*n*" always means the companion document *UdaanHer Saathi MVP: Technical Specification* (`docs/spec.pdf` in the repo); the referenced section contains the exact schema, endpoint, or rule the task implements. Read your task's spec sections *before* writing code.

Rules of the road:

- Work on a branch named `task/T##-short-slug`; open a PR into `main`; someone else reads it before merge. `main` must always start and pass tests.
- A task is not done until its definition-of-done boxes are all checked *by a person other than the author*.
- If a task turns out to conflict with the spec, stop and fix the spec first (one line in `docs/decisions.md` saying what changed and why), then continue.
- Estimates assume a focused beginner-to-intermediate developer with AI coding assistance. If you are 2× over the estimate, ask for help instead of pushing on alone — that is a process feature, not a failure.

Sequencing at a glance. Phases run in order; inside a phase, tasks marked with the same dagger (†) can run in parallel on two machines. Content tasks (Phase 4) need no programming and can start any time after T01 — they are the student-friendly parallel track.

Phase	Outcome	Tasks
0 Foundation	Repo runs on both machines; keys proven	T01–T04
1 Voice loop	Speak to it, it speaks back (no brain yet)	T05–T08
2 Data & protocol	Database and UI-command contract exist	T09–T10
3 Feature 1	Full voice onboarding, profile saved	T11–T14
4 Content	Curriculum, rubrics, visual aids (parallel)	T15–T17
5 Feature 2	Teach → viva → re-teach → progress	T18–T21
6 Demo hardening	Returning users, languages, deploy, rehearsal	T22–T26

2. Phase 0 — Foundation

T01 — Repository scaffold and tooling

Depends on: nothing • **Estimate:** 3 h • **Good for:** anyone on the team

Context (read even if you skip everything else). We have an empty GitHub repository and two laptops with VS Code. This task turns that into a monorepo where backend and frontend both start with one command each, so every later task begins from a working checkout instead of a setup fight. The exact directory tree is prescribed — later tasks name their files against it.

What to do.

1. Clone the empty repo. Create the directory tree from Spec §4 exactly, with empty placeholder files where content comes later.
2. Backend: `pyproject.toml` with pinned deps (`fastapi, uvicorn, pydantic, pydantic-settings, sqlmodel, httpx, langgraph, langchain-openai, openai, pytest, ruff`); create a venv; `uvicorn app.main:app --reload` must start (an empty FastAPI app for now).
3. Frontend: `npm create vite@latest` (React + TypeScript) into `frontend/`; `npm run dev` must serve the default page; add `prettier`.
4. Write `.gitignore` (Python, Node, `.env`, `data/*.db`, `dist/`) and `.env.example` listing every variable in Spec §5 with dummy values.
5. Write `README.md`: what the project is (3 sentences), then the exact commands to run backend, frontend, and tests on a fresh machine.
6. Create `docs/decisions.md` with its first entry: date, "stack pinned per spec v1.0".

Definition of done (all boxes, verifiable by someone else).

- A teammate clones the repo on the *other* laptop and gets backend + frontend running using only `README.md`, no verbal help.
- `git log` shows work on `task/T01-scaffold` merged via PR.
- No real secret appears anywhere in the repo history.

T02 — Keys, config loading, and API smoke tests

Depends on: T01 • **Estimate:** 3 h • **Good for:** anyone; good first task with APIs

Context (read even if you skip everything else). We hold paid credits for Sarvam (speech) and OpenAI (reasoning). Before building anything on top, we prove both keys work from our code, and we build the one config module every later module imports. The exact request formats are in Spec §6 — do not improvise field names.

What to do.

1. Implement `backend/app/config.py` using `pydantic-settings`: loads every Spec §5 variable from `.env`, fails at startup with a clear message if one is missing.
2. Write `backend/scripts/smoke_sarvam.py`: (a) sends a bundled 5-second WAV of someone saying a Gujarati or Hindi sentence to the STT endpoint (Spec §6.1) and prints the transcript; (b) sends the sentence "Namaste, main aapki saathi hoon" to the TTS endpoint (Spec §6.2) with `target_language_code=hi-IN` and writes `out.mp3`. Record the fixture WAV yourselves on a phone.
3. Write `backend/scripts/smoke_openai.py`: one structured-output call — ask the model to return `{"ok": true}` bound to a Pydantic model — and print the parsed object and the model name used.
4. Save the fixture WAV to `backend/tests/fixtures/` for reuse by T05.

Definition of done (all boxes, verifiable by someone else).

- Both smoke scripts run successfully on both laptops with real keys in local `.env`.
- `out.mp3` plays and sounds like natural Hindi.
- Deleting a variable from `.env` makes the backend refuse to start with a message naming that variable.

T03 — Backend skeleton: health, CORS, error shape, latency log †

Depends on: T02 • **Estimate:** 2 h • **Good for:** backend-leaning

Context (read even if you skip everything else). Every endpoint we build later shares three cross-cutting behaviours: a health check for deploys, CORS so the browser may call us, and one uniform error JSON shape (Spec §7 intro). Building them now means no later task reinvents them.

What to do.

1. Implement `GET /health` exactly as Spec §7.1.
2. Add CORS middleware reading `CORS_ORIGINS` from config.
3. Add an exception handler so every unhandled error returns `{"error": {"code", "message"}}` with a 500, and a helper for raising typed 4xx errors in the same shape.
4. Add middleware that logs method, path, status, and elapsed ms for every request.
5. First pytest: `test_health.py` using FastAPI's `TestClient`.

Definition of done (all boxes, verifiable by someone else).

- `curl localhost:8000/health` returns the spec shape; `pytest` passes.
- A deliberately-raised test exception comes back in the uniform error shape, and the log line shows the request.

T04 — Frontend skeleton: kiosk shell †

Depends on: T01 • **Estimate:** 3 h • **Good for:** frontend-leaning

Context (read even if you skip everything else). The learner-facing screen is not a website; it is a kiosk face: one big friendly screen, one large talk button, a content area the agent will fill. This task builds that empty shell and proves the frontend can reach the backend. Design constraints come from the plan's persona: our user reads very little, so everything is large, high-contrast, and image-first.

What to do.

1. Replace the Vite default page with `App.tsx`: full-viewport layout, top 70% content area (empty for now, warm placeholder illustration), bottom 30% a single huge circular "talk" button with a microphone icon, and a small status line ("ready").
2. Plain CSS: minimum 24px body text, large tap targets ($\geq$ 64px), warm palette; must look right on a laptop and a phone (portrait).
3. Implement `src/api.ts` with a typed `getHealth()` calling the backend; show a tiny green/red dot in a corner bound to it.
4. Add `src/types.ts` as an empty placeholder with a comment pointing at Spec §8 (T10 fills it).

Definition of done (all boxes, verifiable by someone else).

- With backend running, the page shows the green dot; with backend stopped, red — no console errors either way.
- On a real Android phone via LAN IP, the layout is usable one-handed and text is legible from arm's length.

3. Phase 1 — The voice loop (walking skeleton)

T05 — STT service module †

Depends on: T02 • **Estimate:** 3 h • **Good for:** backend

Context (read even if you skip everything else). Everything the learner says reaches us as browser audio; this module turns audio bytes into text via Sarvam Saaras v3. It is the only file in the codebase allowed to know Sarvam's STT wire format (Spec §6.1) — callers just see `transcribe(audio_bytes, language) -> TranscriptResult`.

What to do.

1. Implement `app/services/stt.py`: async `transcribe()` using httpx multipart per Spec §6.1, model `saaras:v3`, mode `transcribe`; return a small dataclass (transcript, language_code, request_id, elapsed_ms).
2. Handle failure modes explicitly: non-200 (raise a typed `SttError` carrying Sarvam's message), timeout at 15s, and *empty transcript* (return it as-is; the agent decides what to say — do not raise).

3. Tests: one live test marked `@pytest.mark.live` using the T02 fixture WAV; unit tests with a mocked httpx for the 3 failure modes.

Definition of done (all boxes, verifiable by someone else).

- Live test prints a plausible transcript of the fixture audio in the right script (Gujarati/Devanagari).
- Mocked tests pass; no other module imports httpx for STT purposes.
- `elapsed_ms` is populated (we start watching latency now, per Spec §13).

T06 — TTS service module †

Depends on: T02 • **Estimate:** 3 h • **Good for:** backend

Context (read even if you skip everything else). The mentor's voice. Mirror of T05 for Sarvam Bulbul v3 (Spec §6.2): `synthesize(text, language) -> Mp3Result`. The speaker voice is a config value because choosing the right warm female voice is a product decision we will revisit by ear.

What to do.

1. Implement `app/services/tts.py` per Spec §6.2: JSON body, `model=bulbul:v3`, `speaker` from config, `output_audio_codec=mp3`, sample rate 24000; decode `audios[0]` from base64; return (mp3_bytes, elapsed_ms).
2. Guard the 2500-char limit: if the text is longer, split on sentence boundaries, synthesise sequentially, concatenate — but also log a warning, because per Spec §9.3 mentor replies should be 2–3 sentences and hitting this guard means a prompt is misbehaving.
3. Same failure handling and test pattern as T05. Listen to at least 3 candidate speakers with a Gujarati sentence and record the chosen one in `docs/decisions.md`.

Definition of done (all boxes, verifiable by someone else).

- Live test writes an mp3 that plays natural Gujarati and natural Hindi.
- Mocked failure tests pass; speaker choice + reasoning noted in `docs/decisions.md`.

T07 — `/api/turn` walking skeleton (echo brain)

Depends on: T03, T05, T06 • **Estimate:** 4 h • **Good for:** backend

Context (read even if you skip everything else). Before any AI cleverness, we prove the full pipe: audio in, audio out, under our latency budget. The "brain" in this task is a stub that replies "Aapne kaha: {transcript}" — deliberately trivial, so that when T11 swaps in the real agent, everything around it is already known-good. This task also implements the endpoint's contract (Spec §7.3) which will not change afterwards.

What to do.

1. Implement `POST /api/turn` accepting multipart per Spec §7.3 (accept the `audio` field now; `tapped_option_id` arrives in T10). Session validation is a stub until T09 — accept any `session_id`.
2. Pipeline: STT → echo-stub reply → TTS → response JSON with `transcript`, `reply_text`, `reply_audio_b64`, `ui: {"type": "idle"}`, `stage: "greet"`, and the real `latency_ms` breakdown.
3. Run STT/agent/TTS timings through one small helper so every future node is timed the same way.
4. Test: TestClient posts the fixture WAV (mock Sarvam in unit tests; one live test end to end).

Definition of done (all boxes, verifiable by someone else).

- Live: posting the fixture WAV returns playable mp3 audio of the echo sentence.
- `latency_ms` fields are non-zero and their sum roughly matches wall time.
- Unit tests pass with both Sarvam calls mocked.

T08 — Frontend push-to-talk and playback

Depends on: T04, T07 • **Estimate:** 5 h • **Good for:** frontend

Context (read even if you skip everything else). This is the highest-stakes frontend task: the microphone interaction *is* the product's front door. Push-to-talk (hold to speak, release to send) is chosen over open-mic because it is robust in noisy rooms and unambiguous for a first-time user. After this task, anyone can hold the button, speak, and hear the machine answer — our first demo-able moment.

What to do.

1. Implement `src/audio/recorder.ts`: request mic permission on first press; record `audio/webm;codecs=opus` via `MediaRecorder` while the button is held (pointer events, so touch works); on release, hand back a Blob. Handle permission-denied with a friendly full-screen message.
2. Implement `src/audio/player.ts`: base64 mp3 → `Audio` playback; expose `playing` state; preload and play a soft earcon (`content/assets/earcon.mp3` — any gentle chime, checked into the repo) when a turn is in flight.
3. Wire into `App.tsx` with a visible state machine: `ready` (button breathes) → `listening` (button enlarges, red ring) → `thinking` (earcon + spinner) → `speaking` (waveform-ish animation) → `ready`. Block re-recording while `thinking/speaking`.
4. Show the returned `transcript` small and the `latency_ms` sum in a corner (dev aid; hidden behind a `?debug=1` query flag).

Definition of done (all boxes, verifiable by someone else).

- On laptop and Android phone: hold, say a Gujarati sentence, release, hear the echo reply; all four visual states clearly distinguishable.
- Mic permission denial shows the friendly message, not a broken button.
- 10 consecutive turns work without reload; median round trip logged and reported in the PR description.

4. Phase 2 — Data and protocol

T09 — Database and persistence layer †

Depends on: T03 • **Estimate:** 4 h • **Good for:** backend

Context (read even if you skip everything else). Everything the mentor remembers about a learner lives in five SQLite tables (Spec §10). This task creates them plus a thin repository layer, so agent code never writes SQL. It also implements `delete_learner` — per Spec §12 the delete function must exist from the moment the tables do, because our users' trust is a launch requirement, not a polish item.

What to do.

1. Define the five SQLModel tables exactly per Spec §10; create `data/app.db` on startup if absent.
2. Repository functions: `create_learner, get_learner_by_name_pin` (PIN stored as a salted hash), `save_profile, log_turn, upsert_lesson_progress, upsert_concept_mastery, get_progress(l` (returns the `ProgressPayload` shape of Spec §8), and `delete_learner` (cascades everywhere).
3. Implement `POST /api/session` (Spec §7.2) minus the greeting audio: create a `sessions` row, resolve returning learners by name+PIN, return ids and stage. (T11 adds the spoken greeting.)
4. Implement `GET /api/learner/{id}/progress` (Spec §7.4) over `get_progress`.
5. Tests: full pytest coverage of the repository against a temp database, including delete-cascade and wrong-PIN.

Definition of done (all boxes, verifiable by someone else).

- All repository tests pass; wrong PIN never returns a learner; `delete_learner` leaves zero rows for that learner in every table (asserted).
- `/api/session` round-trips for both new and returning learners via TestClient.

T10 — UI command protocol, both sides †

Depends on: T04 • **Estimate:** 4 h • **Good for:** full-stack; precision matters

Context (read even if you skip everything else). The agent drives the screen by returning one `UICommand` per turn; the frontend is a dumb renderer. This task creates both halves of that contract — Pydantic on the backend, TypeScript on the frontend — from Spec §8, plus the renderer components with placeholder data, so Feature-1 and Feature-2 tasks can emit commands into an already-working screen.

What to do.

1. Backend: `app/models/ui.py` — discriminated-union Pydantic models for all six command types and `ProgressPayload`, exactly per Spec §8.
2. Frontend: `src/types.ts` mirroring them, and a `<Renderer ui={...}/>` switch component.
3. Components with static demo props: `OptionCards` (image cards, huge, tap raises `onTap(id)`), `LessonStep` (image + one-line caption + "step n of m" dots), `VideoEmbed` (YouTube iframe), `ProfileCard`, `ProgressView` (lesson list + concept chips coloured by mastery).
4. Extend `/api/turn` to accept `tapped_option_id` as the alternative to audio (Spec §7.3); a tap skips STT and flows into the same pipeline. Wire `OptionCards` taps to it.
5. Add a hidden dev route/flag that cycles the renderer through one hard-coded example of each command type for visual QA.

Definition of done (all boxes, verifiable by someone else).

- Dev flag shows all six command renders correctly on laptop + phone; captions legible at arm's length.
- Tapping a card sends `tapped_option_id` and the echo brain answers (audio plays) — proving tap-as-a-turn.
- Field-by-field diff of `ui.py` vs `types.ts` vs Spec §8 done by the reviewer.

5. Phase 3 — Feature 1: the mentor gets to know her

T11 — LangGraph mentor skeleton

Depends on: T07, T09 • **Estimate:** 6 h • **Good for:** backend; the most conceptual task

Context (read even if you skip everything else). Now the brain. The mentor is a LangGraph state machine whose stages mirror the pedagogy (Spec §9.1): greet, discover, assess, confirm_profile, teach, viva, reteach, wrapup, close, resume. This task builds the graph's skeleton — state model, router, checkpointing, node scaffolds that produce placeholder replies — and swaps it in for the echo brain. The hard rules in Spec §9.1 (teach unreachable without a profile, etc.) are enforced here in code, not left to prompts.

What to do.

1. Implement `agent/state.py`: the `AgentState` model from Spec §9.2.
2. Implement `agent/graph.py`: one node per stage under `agent/nodes/`, each returning a canned in-character line and a valid `UICommand`; conditional edges per the Spec §9.1 transition table; guard functions for the hard rules; LangGraph SQLite checkpointer keyed by `session_id`.
3. Create `services/llm.py` (ChatOpenAI factory with model + temperature from config) — used by nodes from T12 on.

4. Replace the echo stub in `/api/turn`: load checkpoint, run one graph step with the transcript (or tapped option), persist, return reply/ui/stage. `/api/session` now runs the `greet` (or `resume`) node to produce the real spoken greeting.

5. Tests: drive the graph transcript-by-transcript (no LLM yet) and assert stage sequences, including the guards (e.g. forcing stage to `teach` without a profile must be impossible).

Definition of done (all boxes, verifiable by someone else).

- From the browser: the mentor speaks first, and successive turns visibly advance stages (stage shown under `?debug=1`).
- Killing and restarting the backend mid-session preserves the conversation (checkpoint proof).
- Transition tests pass, including both guard rules.

T12 — Onboarding conversation: greet, discover, assess, confirm

Depends on: T11 • **Estimate:** 8 h • **Good for:** backend + prompt writing; the heart of Feature 1

Context (read even if you skip everything else). This task replaces four placeholder nodes with the real thing: a warm 5–7 minute voice conversation, in the session language, that ends with a saved learner profile. Persona and prompting rules are Spec §9.3 — the mentor is a patient didi, one question at a time, never test-like. Consent (Spec §12) happens in `greet`: she must agree aloud before anything about her is stored. Assessment infers a starting level from stories, not quiz answers.

What to do.

1. Write the four stage prompts in `agent/prompts/` per Spec §9.3, each stating: role, session language, what this stage must learn, the 2–3-sentence reply cap, and the structured output expected.

2. `greet`: welcome, ask name, ask consent plainly ("Shall I remember you, so next time we continue where we left?"); on refusal set a no-persist flag honoured everywhere downstream.

3. `discover`: village and current work by voice; interests via a `show_options` command with 4 skill cards (from curriculum titles); accept tap or voice.

4. `assess`: 3–4 story-questions about the chosen skill, adapted to her answers; then a structured-output call producing `ProfileDraft` (name, village, interest, starting_level, notes) — bind to Pydantic, never parse prose.

5. `confirm_profile`: emit `show_profile_card` *and* read the profile aloud; on yes, save via T09 repository (with `consent_given_at`); on correction, apply the fix and re-confirm.

6. Tests: mocked-LLM tests asserting each prompt receives the right context and each structured output lands in state; plus a scripted live run in Hindi and one in Gujarati, transcripts saved to `docs/transcripts/` for prompt review.

Definition of done (all boxes, verifiable by someone else).

- Live in the browser, in Gujarati: a brand-new visitor is greeted, consents, chats, taps or says an interest, answers stories, hears her profile read back, confirms — and a correct `learners` row exists.
- Saying "no" to consent still allows the conversation, and the database stays empty afterwards (asserted).
- Every mentor reply in the two recorded transcripts is ≤ 3 sentences and contains no forbidden words (test/exam/score/wrong); reviewer reads both transcripts and signs off on tone.

T13 — Feature-1 frontend integration and voice-first polish

Depends on: T10, T12 • **Estimate:** 4 h • **Good for:** frontend

Context (read even if you skip everything else). Onboarding now works; this task makes it feel right on screen: commands rendering mid-conversation, option cards answerable by tap or voice interchangeably, and the visual state machine from T08 staying truthful during multi-second agent turns. This is where "the agent controls the software" becomes visible to a judge.

What to do.

1. Wire real `UICommand`s from `/api/turn` through the T10 renderer during a live onboarding; fix layout/timing issues (command should render when the mentor *starts* speaking, not after).
2. Language picker on the very first screen only: three big flag-free buttons, each labelled in its own script (Gujarati, Hindi, English) feeding `/api/session`.
3. Add gentle auto-hints: if `ready` persists 15s after a question, replay the mentor's last audio.
4. Record a full onboarding as a screen capture; file issues for anything janky and fix the top three.

Definition of done (all boxes, verifiable by someone else).

- A non-team-member (friend/family) completes onboarding in Hindi or Gujarati with zero verbal coaching from you — observed, notes taken, filed.
- Cards can be answered by voice ("beauty ka kaam") *or* tap with identical downstream behaviour.
- Screen capture of a clean run checked into `docs/` (or linked) for the demo archive.

T14 — Feature-1 milestone gate and seeded returning learner

Depends on: T13 • **Estimate:** 2 h • **Good for:** whole team, together

Context (read even if you skip everything else). A deliberate stop-and-verify point: Feature 1 is declared done here against the acceptance criteria, and we create the demo fixture everybody downstream relies on — the pre-seeded learner "Sunita" with a completed profile, ready to start lesson one.

What to do.

1. Run acceptance items 1 and 2 of Spec §15 end to end on both laptops and one phone; log outcomes in `docs/decisions.md`.
2. Write `scripts/seed_demo.py`: idempotently creates learner Sunita (Gujarati, tailoring, `starting_level=some` PIN 1234) with no lesson progress.
3. Fix whatever the gate run surfaces before starting Phase 5 coding.

Definition of done (all boxes, verifiable by someone else).

- Spec §15 items 1–2 pass, witnessed by both team members.
- `seed_demo.py` runs twice without duplicating Sunita; name+PIN login greets her by name.

6. Phase 4 — Content (parallel track, no coding)

T15 — Tailoring curriculum: three lessons †

Depends on: T01 (repo only) • **Estimate:** 8 h spread over days • **Good for:** Aishi / non-programmer; domain care over code

Context (read even if you skip everything else). The agent never invents curriculum; it personalises from a hand-built JSON file (Spec §11.1). This task writes that file for tailoring: three lessons, each with 3–5 concepts and 4–6 teachable steps with images. Ground truth is the NSDC Qualification Pack for assistant tailor / sewing machine operator (`https://nqr.gov.in`) — free, government-standard. Quality bar: a human tailor reading a lesson's teaching notes would nod.

What to do.

1. Read the relevant NSDC QP; choose three first lessons a beginner-to-intermediate learner needs (suggested: taking body measurements; fabric grain and cutting basics; straight-stitch seams and finishing). Record the choice + QP reference in `docs/decisions.md`.
2. For each lesson fill the Spec §11.1 schema: concepts (with `must_land` flags), steps with `teaching_notes` written *for the LLM* in plain English (what to convey, what to emphasise, what mistakes to warn about), captions in en/hi/gu.
3. Source or draw one clear image per step (simple line diagrams beat photos; note licences) into `content/assets/`.
4. Ask a real tailor (local, family, or a detailed tutorial cross-check) to review lesson 1's notes for correctness.

Definition of done (all boxes, verifiable by someone else).

- `tailoring.json` validates against the loader (T18 wires it; until then, against the schema by inspection with a teammate).
- Every step has an existing image file; every caption exists in all three languages.
- Tailor-review of lesson 1 happened; corrections applied and noted.

T16 — Viva rubrics for the three lessons †

Depends on: T15 (lesson concepts fixed) • **Estimate:** 5 h • **Good for:** Aishi / non-programmer

Context (read even if you skip everything else). The viva is only as fair as its rubric (Spec §11.2). For each concept: 2–3 conversational questions, plus examples of what a *correct* answer sounds like from a non-literate learner — spoken, informal, code-mixed ("pehle naap lena, phir kapda katna") — and what common confusion sounds like. These exemplars are the LLM's grading context; without them it grades like a schoolteacher, which is exactly what we must not be.

What to do.

1. For every concept in T15's three lessons, write 2–3 questions phrased as friendly conversation ("Suppose the blouse comes out tight at the shoulder — what would you check?").
2. For each question: 2–3 `sounds_right` spoken-style exemplars and 1–2 `sounds_confused` ones, in Hindi/Gujarati-flavoured informal phrasing.
3. Fill `tailoring_rubrics.json` per Spec §11.2; have the other team member try to answer each question aloud and check the exemplars cover natural answers.

Definition of done (all boxes, verifiable by someone else).

- Every T15 concept has ≥ 2 questions with both exemplar kinds; JSON is valid.
- Read-aloud trial done; at least three exemplars were revised because of it (if none were, the trial was not honest).

T17 — Visual-aid index †

Depends on: T15 (concept ids fixed) • **Estimate:** 4 h • **Good for:** Aishi / non-programmer

Context (read even if you skip everything else). When re-teaching, the agent may only pick from a hand-vetted list of videos and diagrams (Spec §11.3) — curated beats searched, because a live YouTube search in a demo is a coin flip. Your judgement here *is* the feature.

What to do.

1. For each T15 concept, find ≥ 2 aids: a good Hindi (or Gujarati, rarer) YouTube tutorial segment and/or a clear diagram. Watch every video fully; check it is correct, respectful, and beginner-paced.
2. Fill `index.json` per Spec §11.3 with a one-line `note` on why each was chosen; convert YouTube links to embed form (`youtube.com/embed/<id>`).
3. Verify each embed URL actually plays inside an iframe (some videos disable embedding — replace those).

Definition of done (all boxes, verifiable by someone else).

- Every concept has ≥ 2 entries; every video watched end to end by a human; every embed URL tested in an iframe.
- No entry relies on live search; language tags correct.

7. Phase 5 — Feature 2: the mentor teaches and checks

T18 — Content loader and the teach stage

Depends on: T11, T15 • **Estimate:** 6 h • **Good for:** backend

Context (read even if you skip everything else). The teach node walks Sunita through a lesson one step at a time: each turn shows one `show_lesson_step` command while the mentor narrates that step from its `teaching_notes`, and she can interrupt with questions at any point. Lesson choice comes from her profile and progress (first incomplete lesson). Content must be validated at startup — a typo in JSON must fail the boot, not the demo (Spec §11).

What to do.

1. Implement `content/loader.py`: Pydantic models mirroring Spec §11.1–11.3, loaded and validated at startup; helpful error naming file + field on failure.
2. Implement the `teach` node: pick lesson (profile + `lesson_progress`); per turn, narrate the current step (LLM with the step's `teaching_notes` + persona prompt) and emit its `show_lesson_step`; detect from her utterance whether she asks a question (answer it, stay on step), wants to continue (advance), or wants to stop (go to `wrapup`).
3. Mark `lesson_progress = in_progress` on entry; transition to `viva` after the last step.
4. Tests: loader failure cases (missing image path, missing language key); mocked-LLM step-advance logic; one live run of lesson 1 with transcript saved.

Definition of done (all boxes, verifiable by someone else).

- Corrupting `tailoring.json` makes the backend refuse to start with a message naming the bad field.
- Live: Sunita (seeded) hears lesson 1 step by step with images changing; an interruption ("video dhime chalao jaisa kuch poocho" — ask anything) gets answered without losing the step position.
- Reaching the last step lands in `viva` (visible in debug stage).

T19 — The viva stage: conversational check with rubric grading

Depends on: T18, T16 • **Estimate:** 6 h • **Good for:** backend; second-most-delicate prompts

Context (read even if you skip everything else). After the lesson, the mentor "chats about what we did": one rubric question at a time, graded `strong`/`shaky` per concept using the T16 exemplars as grading context (Spec §11.2). Framing is everything — Spec §9.3's forbidden-word list applies doubly here. Grades are written to `concept_mastery`; the routing decision (reteach vs wrapup) is computed in code from the grades, never by asking the LLM "should we reteach?".

What to do.

1. Implement the `viva` node: select the next unasked question covering a `must_land` concept first; ask it conversationally; on her answer, run a structured-output grading call (question, her transcript, the exemplars) returning `{concept_id, grade, one_line_reason}` at temperature ≤ 0.2.
2. After each grade: encouraging micro-feedback in-language (grade never spoken as a grade); write `concept_mastery`; when all lesson concepts are covered, route: any `shaky` → `reteach`, else → `wrapup`.
3. Tests: grading calls with fabricated strong/confused answers built from the rubric exemplars — assert expected grades (mocked and 6-case live); routing unit tests.

Definition of done (all boxes, verifiable by someone else).

- Live: answering two questions well and one badly yields matching `concept_mastery` rows and routes to `reteach`.
- In the recorded transcript the mentor never utters test/exam/score/wrong/fail in any session language; reviewer signs off on warmth.
- The 6-case live grading check gets ≥ 5 right; misses analysed in the PR description.

T20 — The re-teach loop

Depends on: T19, T17 • **Estimate:** 5 h • **Good for:** backend

Context (read even if you skip everything else). The product's soul (plan document §8.2): where she is shaky, the mentor immediately explains *differently* — new analogy, a diagram, or a curated video — then gently re-checks with one question. Code-enforced limits from Spec §9.1: aids come only from the T17 index; max 2 re-teach rounds per concept, then the concept is marked "revisit next session" and the mentor moves on warmly.

What to do.

1. Implement the `reteach` node: take the first shaky concept; choose an aid from the index (prefer session language, fall back to Hindi — log when fallback happens); explain differently (prompt gets her original wrong answer so the explanation targets *her* confusion); emit `show_video` or `show_lesson_step` with the diagram.
2. Re-check with one fresh rubric question (not the one she missed); regrade; update mastery and `reteach_counts`; loop or move on per the limits.
3. Tests: aid-selection fallback logic; the 2-round cap (third round impossible, concept flagged for next session); one live run with a deliberately-wrong answer.

Definition of done (all boxes, verifiable by someone else).

- Live: a deliberately-wrong viva answer visibly triggers a different explanation with a video or diagram on screen, then a re-check.
- Forcing two failed rounds ends in a warm move-on (transcript reviewed) and a "revisit" flag in the database — never a third drill.
- Aid selection never returns anything outside `index.json` (test asserts this).

T21 — Wrap-up, progress view, and Feature-2 gate

Depends on: T20 • **Estimate:** 4 h • **Good for:** full-stack, then whole team

Context (read even if you skip everything else). The session must end the way a good lesson does: here is what you did, here is what comes next. The `wrapup` node emits `show_progress` (data from T09's `get_progress`) while the mentor narrates it; `close` persists everything and says goodbye. Then the whole team runs the Feature-2 acceptance gate.

What to do.

1. Implement `wrapup`: mark lesson completed when all `must_land` concepts are strong (else leave `in_progress` with the revisit flags); emit `show_progress`; narrate it simply ("two things solid, one we will look at again").
2. Implement `close`: final save, warm goodbye, session `ended_at`.
3. Frontend: make sure `ProgressView` renders the real payload well (concept chips coloured by mastery, next-step line).
4. Gate: run Spec §15 items 3 and 6 end to end with Sunita; log in `docs/decisions.md`; fix blockers.

Definition of done (all boxes, verifiable by someone else).

- Spec §15 item 3 passes end to end, witnessed by both team members (lesson → viva → forced re-teach → accurate progress screen).
- `delete_learner` on a test learner verified again post-Feature-2 (item 6).
- Progress screen matches database contents exactly for Sunita after the run.

8. Phase 6 — Demo hardening

T22 — Returning-user flow and session resume †

Depends on: T14, T21 • **Estimate:** 4 h • **Good for:** full-stack

Context (read even if you skip everything else). The demo's "it remembers her" beat: Sunita comes back, gives her name and speaks her 4-digit PIN, and the mentor greets her by name with her history and continues. The `resume` stage exists in the graph since T11; this task makes it real and builds the minimal first-screen flow around it.

What to do.

1. First screen after language pick: two big buttons, spoken aloud — "I am new" / "I have come before". Returning: mentor asks name by voice, then PIN by voice; STT digits parsed leniently (words or digits, any of the three languages); 3 failed attempts falls back gracefully to starting fresh with an apology.
2. Implement `resume`: load profile + progress; greeting references the last completed thing ("Last time we finished measuring; today, cutting"); jump into `teach` at her next lesson.
3. Tests: PIN parsing table-test ("one two three four", `1234`, hindi/gujarati number words); resume greeting content assembled from fixture data.

Definition of done (all boxes, verifiable by someone else).

- Live: full return visit — name, spoken PIN, personalised greeting naming her actual last lesson, teaching resumes at the right lesson.
- Wrong PIN 3× degrades politely to the new-visitor flow; no dead end anywhere.

T23 — Language sweep: gu / hi / en †

Depends on: T21 • **Estimate:** 4 h • **Good for:** whole team; testing-heavy

Context (read even if you skip everything else). The architecture claims a language is configuration (Spec §13). This task proves it: every flow, in all three shipped languages, hunting for hard-coded strings, wrong-language TTS, and prompts that drift into English mid-conversation.

What to do.

1. Run onboarding + a lesson + viva in each of `gu-IN`, `hi-IN`, `en-IN`; log every string that appears in the wrong language (screen or voice) and every prompt drift; fix them (usually: missing language key in content JSON, or a prompt missing the explicit target-language line).
2. Grep the codebase for hard-coded user-facing strings; move any into content maps.
3. Confirm the visual-aid Hindi-fallback line is spoken when a Gujarati aid is missing ("This video is in Hindi — shall we watch it together?").

Definition of done (all boxes, verifiable by someone else).

- One full session per language recorded; zero wrong-language utterances in the final recordings.
- Grep shows no user-facing literals outside content/ and prompts/.

T24 — Latency and resilience polish

Depends on: T21 • **Estimate:** 5 h • **Good for:** backend-leaning full-stack

Context (read even if you skip everything else). Spec §15 demands median turn latency $\leq$ 5s and graceful failure. We have logged `latency_ms` since T07; now we act on the data. The biggest levers, in order: shorter LLM replies (prompt discipline), running the TTS call as soon as reply text exists, model choice for grading calls, and never letting one slow external call hang the UI forever.

What to do.

1. Pull the latency log; identify the slowest component per stage; apply the levers above; re-measure 20-turn medians before/after and put the table in the PR.
2. Timeouts + one retry (with jitter) on Sarvam and OpenAI calls; on final failure, the mentor *speaks* a graceful line from a small set of pre-synthesised local mp3s ("My ears failed for a moment — say that again?") — never a silent error toast for the learner.
3. Verify the earcon covers the whole thinking gap and the button state machine never deadlocks after an error (button returns to `ready`).

Definition of done (all boxes, verifiable by someone else).

- 20-turn median $\leq$ 5s on the dev machine over real APIs; before/after table in the PR.
- Killing Wi-Fi mid-turn produces the spoken apology and a recoverable UI; turning Wi-Fi back on, the next turn works without reload.

T25 — Deployment to a public URL

Depends on: T22, T23, T24 • **Estimate:** 5 h • **Good for:** whoever did T01

Context (read even if you skip everything else). The demo runs from a public URL, not localhost. Recommended: backend on Render (Python web service, persistent disk for SQLite, env vars from Spec §5) and frontend on Vercel — or Render alone serving the built frontend as static files from FastAPI, which is one moving part fewer; choose and record in `docs/decisions.md`. Note: browsers require HTTPS for microphone access away from localhost — both platforms give it by default; do not deploy anywhere that does not.

What to do.

1. Deploy backend with env vars set in the platform dashboard (never in the repo); persistent disk mounted at `data/`; run `seed_demo.py` once via the deploy shell.
2. Deploy frontend with the backend URL in its build env; set `CORS_ORIGINS` to the frontend origin.
3. Full smoke on the deployed URL from a phone on *mobile data* (not your Wi-Fi): mic permission, onboarding start, one lesson step.
4. Write `docs/deploy.md`: how to redeploy, where the env vars live, how to re-seed.

Definition of done (all boxes, verifiable by someone else).

- The public URL passes the T08-style 10-turn test from a phone on mobile data.
- A teammate redeploys a trivial change using only `docs/deploy.md`.
- No secret in the repo; keys only in platform dashboards.

T26 — Demo rehearsal, backup plan, and final gate

Depends on: T25 • **Estimate:** 4 h + rehearsals • **Good for:** whole team

Context (read even if you skip everything else). Competitions are lost to microphones and venue Wi-Fi, not to missing features. This task runs the full acceptance checklist (Spec §15), rehearses the 5-minute script from the plan document (open in Gujarati → live onboarding → cut to seeded Sunita → lesson + deliberate wrong answer → visible re-teach → progress + roadmap slide), and prepares the fallback for when the venue betrays us.

What to do.

1. Run every Spec §15 item on the deployed URL, laptop + phone; fix or consciously accept (in `docs/decisions.md`) every failure.
2. Rehearse the script twice with a timer; assign who speaks and who drives; script the deliberate wrong answer word-for-word.
3. Backup plan: (a) full screen recording of a perfect run, downloaded locally; (b) phone hotspot tested as network fallback; (c) a wired/known-good microphone; (d) local run instructions if the venue blocks everything.
4. Reset demo data: re-run `seed_demo.py`, delete rehearsal learners.

Definition of done (all boxes, verifiable by someone else).

- All Spec §15 boxes checked on the deployed URL, dated in `docs/decisions.md`.
- Two timed rehearsals done, both $\leq$ 5 minutes; backup video exists on two devices; hotspot fallback tested end to end.
- Database contains exactly the seeded demo state and nothing else.

9. After the MVP (parking lot, do not start)

Marathi and Bengali enablement; the marketplace profile view; DSPy optimisation of the viva-grading prompts against logged transcripts; streaming TTS for sub-2s perceived latency; facilitator dashboard; kiosk packaging (fullscreen tablet browser). Each becomes a task book of its own only after the competition.